

交通大数据

集成学习

- 刘志远教授
- Email: zhiyuanl@seu.edu.cn



聚类分析

□ 学习目标

- 了解集成学习的分类
- 掌握典型Boosting和Bagging集成方法
- 了解集成学习中模型的多样性



1 集成学习分类

- 集成学习的定义

- 集成学习分类



集成学习 (Ensemble learning)

请大家思考以下场景

- 在日常生活中，我们经常需要做出决策，比如选修课程。**各位同学面对没有学习过的课程时，如何判断这门课程是否适合自己？**
 - 大家通常会征询学长学姐的上课体验，**结合多个意见，做出更好的决策**



集成学习 (Ensemble learning)

请大家思考以下场景

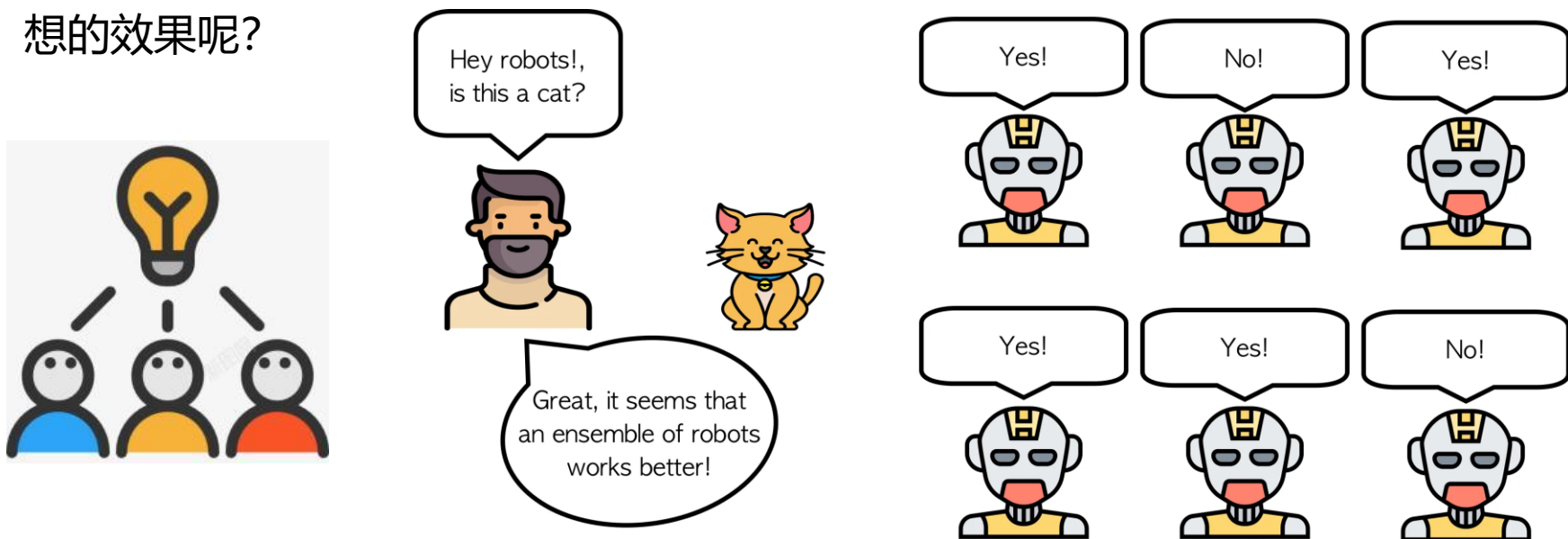
- 在团队合作参加竞赛时，你们是如何分配工作的？
 - 每个人都有自己的专长和知识，**结合多个人的意见，贡献和特长**，可以产出更好的成果



**我们在面对决策时，往往习惯于综合多方视角与多方信息，
做出最优决策！**

集成学习 (Ensemble learning)

相信大家都有这样的经历，在做重要决定时，往往会参考或综合多个朋友或家人的意见，集思广益，而不是独断专行。对于机器学习过程亦是如此，不同的学习器对于不同的数据环境和学习目标，有不同的学习效果，如果我们**在构建机器学习模型的过程中，能够结合不同学习器的优势来共同完成同一个任务**，是不是会取得更理想的效果呢？

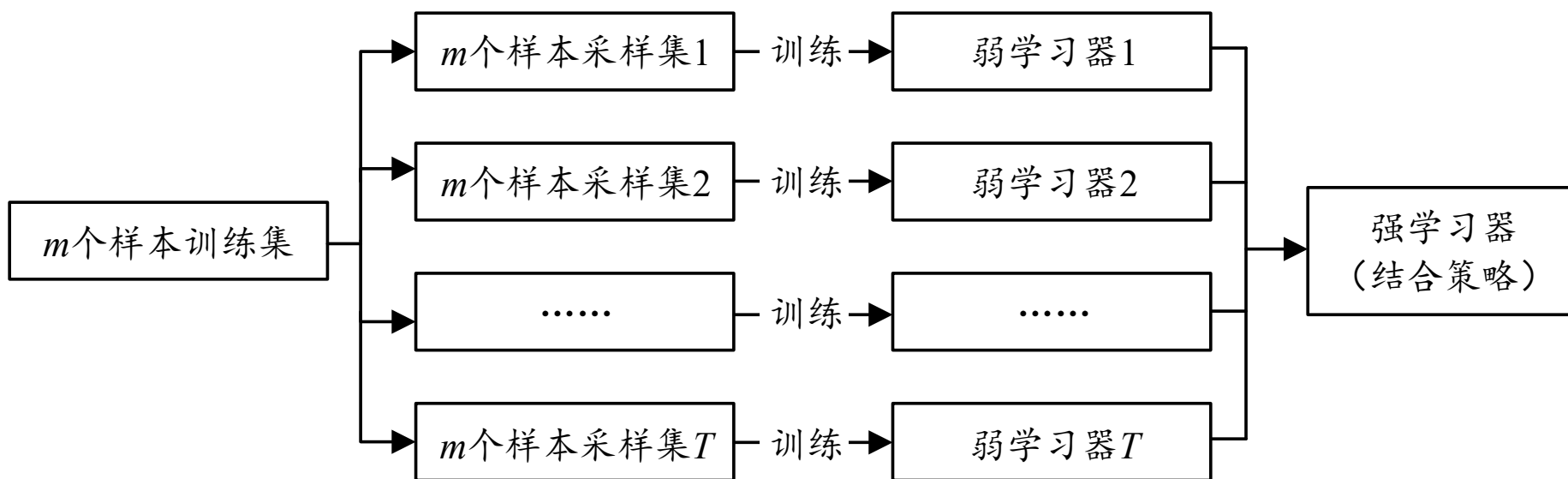


集成学习通过把多种学习器组合在一起，共同协作形成一个具有较强性能的集成学习器（更加精准、更加稳定）



集成学习分类

集成学习通过将多个学习器组合来实现提升模型性能的目标。通常，集成学习模型由两个学习步骤构成：**首先构建一系列个体学习器，随后定义一种组合策略将这些个体学习器进行集成。**个体学习器一般可以是任意机器学习模型，例如决策树模型、支持向量机模型、神经网络模型。



集成学习分类

根据个体学习器的不同，集成学习的集成方式可以分为两类：**同质集成和异质集成**。

同质集成：个体学习器是同质的（homogeneous），即属于同种类型或拥有相似的性能，此时的个体学习器也被称为**基学习器**（base learner），相应的学习算法是基学习算法（base learning algorithm）。

异质集成：个体学习器是异质的（heterogeneous），此时的个体学习器也被称为**组件学习器**（component learner），个体学习器由不同学习算法训练生成

目前流行的集成算法大多是同质集成，以决策树或者神经网络作为个体学习器构建集成学习模型。**本课主要介绍两类经典的同质集成方法：Boosting与 Bagging**



2 Boosting 算法

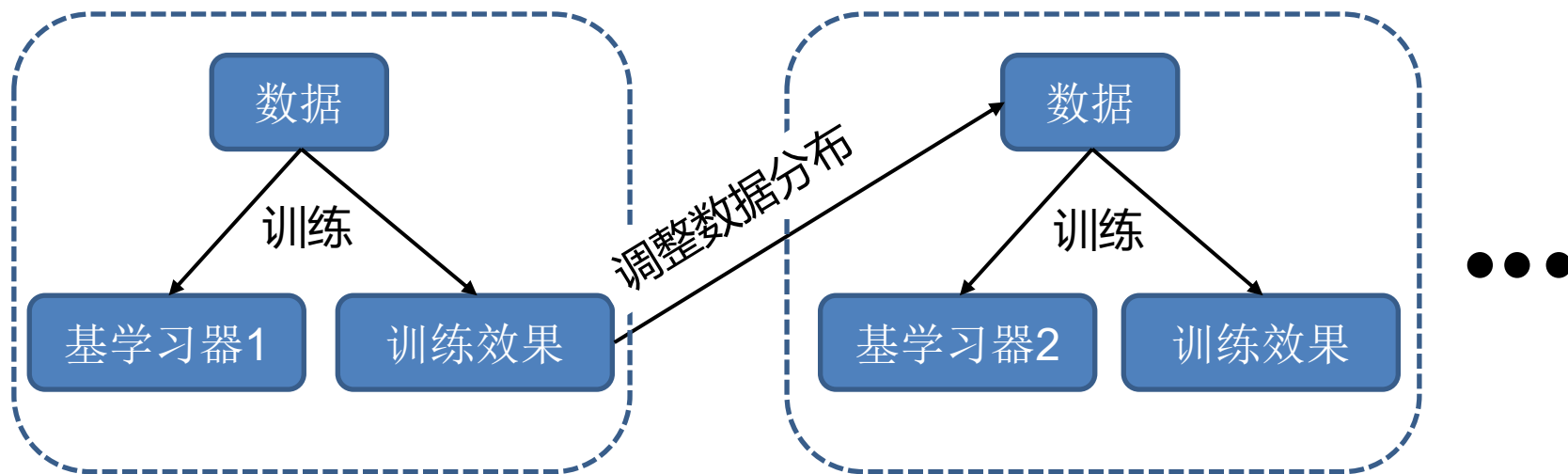
- Boosting简介
- AdaBoost算法
- 提升树
- 梯度提升树



Boosting 简介

集成学习中，通常称泛化性不高的单一基学习器为弱学习器（weak learner）。尽管单一弱学习器的性能十分有限，但通过将多个弱学习器进行结合，往往能够获得巨大的性能提升。**Boosting算法便旨在将多个预测性能较弱的基学习器顺序组合，形成一个强学习器。**

Boosting算法的基本思想是：先从初始训练集训练出一个基学习器，**再根据基学习器的表现对训练样本分布进行调整，使得先前基学习器做错的样本受到更多的关注，**然后基于调整后的样本集训练下一个基学习器。



Boosting 简介

Boosting算法的基本流程可以概括为如下步骤：

- I. 设计一个有一定预测能力的基学习器；
- II. 使用基学习器对训练集进行预测；
- III. 根据预测结果调整训练样本的分布，使得先前基学习器做错的训练样本在后续受到更多关注；
- IV. 在调整分布后的训练集上，再训练一个新的基学习器，即返回步骤 I。

反复执行以上步骤，直到学习器达到设定的预测精度或达到最大迭代次数。

最终的集成模型即为这些基学习器的结合（例如对基学习器预测结果线性加权求和）。



Boosting 简介

基于Boosting算法的设计思路，请思考：Boosting算法主要降低模型的偏差还是方差？为什么？



Boosting 简介

Boosting算法流程中第 III 步**调整训练集样本分布**通常有两种处理方式：

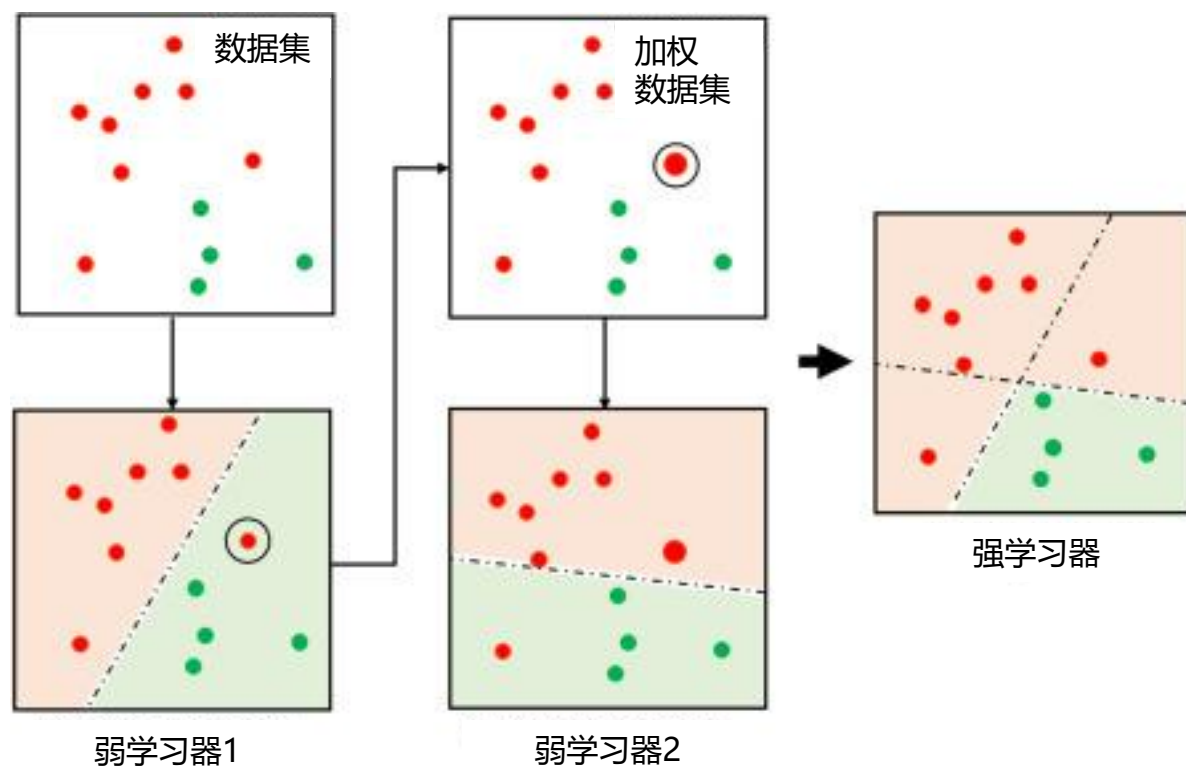
- **重赋权法 (reweighting)**：在每一轮基学习器训练完成后，根据基学习器对于各个样本的预测误差，提高分类错误样本的权重，并降低分类正确样本的权重
- **重采样法 (resampling)**：并非所有机器学习模型都可以处理带权重的样本，对于这类模型，可以采用重采样法更新样本权重，重采样法并不直接将带权样本交给学习器，而是**按照样本权重对训练集进行加权随机采样**，生成一个新的数据集作为下一轮训练的训练集

Boosting算法在训练的每一轮都要检查当前生成的基学习器是否满足基本条件，例如：检查当前基分类器是否比随机猜测更好。若条件不满足，则意味着数据中已没有更多有价值的信息可供学习，故弃用当前的基学习器，并终止学习过程，最终仅对此前迭代中的基学习器进行集成。



AdaBoost

AdaBoost是最具代表性的一类Boosting算法，在训练弱分类器的过程中，AdaBoost算法使用重赋权法，不断调整样本的分布，在每一轮迭代中都更加关注前一轮迭代分类错误的样本。**经过多轮迭代，Adaboost训练出大量弱分类器，并将它们进行加权线性组合，最终形成一个强分类器。**



AdaBoost

以二分类问题为例说明AdaBoost的算法步骤:

设训练集为 $\{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$, 其中 x_i 是输入的训练样本, 其标签 $y_i \in \{-1, +1\}$ 。训练集共包含 m 个样本, 其中正样本数 m^+ , 负样本数 m^- 。记第 t 轮迭代中第 i 个样本的权重为 $w_{i,t}$ 。在第一轮迭代中, $w_{i,1} = 1/m, i \in \{1, \dots, m\}$ 。

基于预设的最大基分类器数量 T , AdaBoost一共进行 T 轮迭代。在第 t 轮迭代中, AdaBoost会训练一个新的基分类器, 记该分类器对第 i 个样本的预测结果为 $h_t(x_i)$,

AdaBoost计算分类器在训练集上的错误率

$$\varepsilon_t = \sum_{i=1}^m w_{t,i} I(h_t(x_i) \neq y_i)$$

其中 $I(\cdot)$ 是指示函数, $I(x) = \begin{cases} 1, & x \text{为真} \\ 0, & x \text{为假} \end{cases}$, 在上式中, 对于预测正确的样本, 该指示函数

数值为0, 对于预测错误的样本, 指示函数值为1



AdaBoost

为表示分类器 h_t 在最终的集成分类器中的**重要程度**，引入系数 $\alpha_t = \frac{1}{2} \ln \left(\frac{1}{\varepsilon_t} - 1 \right)$ 。容易看出，分类错误率越小的分类器，其**重要度** α_t 越高。若分类器预测**错误率** ε_t 低于0.5，那么 $\alpha_t > 0$ ；若 ε_t 不低于0.5，见课件18页的讨论；若 ε_t 为0，可以提前结束训练过程。基于训练结果，AdaBoost通过如下步骤计算调整第 i 个样本的权重：

$$\begin{aligned}\tilde{w}_{t+1,i} &= w_{t,i} \beta_{t,i} \\ \beta_{t,i} &= \exp(-\alpha_t y_i h_t(x_i))\end{aligned}$$

$\alpha_t > 0$ 时，

若样本预测正确， $y_i h_t(x_i) > 0$ ，则 $-\alpha_t y_i h_t(x_i) < 0$ ，此时权重系数 $\beta_{t,i}$ 小于1

若样本预测错误， $y_i h_t(x_i) < 0$ ，则 $-\alpha_t y_i h_t(x_i) > 0$ ，此时权重系数 $\beta_{t,i}$ 大于1

新一轮迭代的样本权重由 $\tilde{w}_{t+1,i}$ 归一化得到：

$$w_{t+1,i} = \frac{\tilde{w}_{t+1,i}}{\sum \tilde{w}_{t+1,i}}$$



AdaBoost

最终集成分类器的**预测结果**由各个基分类器预测结果的加权线性组合投票决定，结合方法如下式所示（ T 是分类器的总数量）：

$$H_{final}(\mathbf{x}) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(\mathbf{x}) \right)$$

其中， $H_{final}(\cdot)$ 是最终获得的集成分类器， $\mathbf{x}$ 是该分类器的输入特征



AdaBoost

在AdaBoost算法中，系数 $\alpha_t = \frac{1}{2} \ln \left(\frac{1}{\varepsilon_t} - 1 \right)$ ，表示分类器 h_t 在最终的集成分类器中的

重要程度。AdaBoost模型默认分类错误率低于0.5，然而实践中存在分类预测错误率 ε_t 不低于0.5的情况，此时分类器权重系数 $\alpha_t < 0$ 。对于这类情况，**常有以下处理方式**：

1. 结束迭代；
2. 重新训练弱学习器（适合训练数据选取过程中存在随机性的情况）
3. 不做任何特殊处理，将该弱学习器正常加入集成模型（可以理解为该弱学习器效果太差，因此在集成模型中做“反方向”的预测，**部分实践**表明这种处理效果很好）

备注：对于二分类问题，弱分类器的分类错误率通常低于0.5（优于随机选择），对于多分类问题，弱分类器的错误率高于0.5的情况比较多见，现有很多研究对处理多分类问题的AdaBoost算法进行设计，感兴趣的同学可以自行查找资料学习。



试一试

给定如表所示训练数据，只有一个特征。假设弱分类器是需要确定一个阈值 v ，即由 $x < v$ 或 $x > v$ 来决定分类结果（1或-1）。试用AdaBoost算法，学习一个强分类器（使其在训练集上误差为0）。

序号	1	2	3	4	5	6	7	8	9	10
x	0	1	2	3	4	5	6	7	8	9
y	1	1	1	-1	-1	-1	1	1	1	-1



试一试

给定如表所示训练数据，只有一个特征。假设弱分类器是需要确定一个阈值 v ，即由 $x < v$ 或 $x > v$ 来决定分类结果（1或-1）。试用AdaBoost算法，学习一个强分类器（使其在训练集上误差为0）。

序号	1	2	3	4	5	6	7	8	9	10
x	0	1	2	3	4	5	6	7	8	9
y	1	1	1	-1	-1	-1	1	1	1	-1

1. 初始化权值

$$\mathbf{w}_1 = (w_{11}, w_{12}, \dots, w_{110})$$
$$w_{1i} = 0.1, \quad i = 1, 2, \dots, 10$$



试一试

2. $t=1$ 时，当前权值分布 w_1 下，通过计算，错误率最低的阈值为2.5，即

$$h_1(x) = \begin{cases} 1, & x < 2.5 \\ -1, & x > 2.5 \end{cases}$$

此时 $h_1(x)$ 的误分类样本为 $x = 6、7、8$ 的样本。误差率为 $\varepsilon_1 = w_{7i} + w_{8i} + w_{9i} = 0.3$ ，

计算系数 $\alpha_1 = \frac{1}{2} \log \left(\frac{1}{\varepsilon_1} - 1 \right) = 0.4236$ 。

此时， $H_1(x) = 0.4236h_1(x)$ 分类器 $\text{sign}[H_1(x)]$ 在训练数据集上有3个误分类点，分别是 $x = 6、7、8$ 的数据点

序号	1	2	3	4	5	6	7	8	9	10
x	0	1	2	3	4	5	6	7	8	9
y	1	1	1	-1	-1	-1	1	1	1	-1
$H_1(x)$	0.42	0.42	0.42	-0.42	-0.42	-0.42	-0.42	-0.42	-0.42	-0.42
$\text{sign}(H_1(x))$	1	1	1	-1	-1	-1	-1	-1	-1	-1



试一试

2. $t=1$ 时, 调整权重, $\mathbf{w}_2 = (w_{21}, w_{22}, \dots, w_{210})$,

$$w_{2j} = \frac{w_{1j} \exp(-\alpha_1 y_j h_1(x_j))}{\sum_{i=1}^m w_{1i} \exp(-\alpha_1 y_i h_1(x_i))}, j = 1, 2, \dots, 10$$

得到:

$$\mathbf{w}_2 = (0.07143, 0.07143, 0.07143, 0.07143, 0.07143, 0.07143, \mathbf{0.16667}, \mathbf{0.16667}, \mathbf{0.16667}, 0.07143)$$

可以看出, 在下一轮的训练中, $x = 6、7、8$ 的数据点有更高的权重, 而在本轮训练中被正确识别的样本权重有所降低

序号	1	2	3	4	5	6	7	8	9	10
x	0	1	2	3	4	5	6	7	8	9
y	1	1	1	-1	-1	-1	1	1	1	-1
$H_1(x)$	0.42	0.42	0.42	-0.42	-0.42	-0.42	-0.42	-0.42	-0.42	-0.42
$\text{sign}(H_1(x))$	1	1	1	-1	-1	-1	-1	-1	-1	-1



试一试

3. $t=2$ 时, 当前权值分布 w_2 下错误率最低的阈值为8.5, 即

$$h_2(x) = \begin{cases} 1, & x < 8.5 \\ -1, & x > 8.5 \end{cases}$$

$h_2(x)$ 的误分类样本为 $x = 3, 4, 5$ 的数据点, 误差率为 $\varepsilon_2 = w_{24} + w_{25} + w_{26} = 0.2143$ 。

计算系数 $\alpha_2 = \frac{1}{2} \log \left(\frac{1}{\varepsilon_2} - 1 \right) = 0.6496$

此时 $H_2(x) = 0.4236h_1(x) + 0.6496h_2(x)$, 此时分类器 $\text{sign}[H_2(x)]$ 有3个误分类点, 分别是 $x = 3, 4, 5$ 对应的数据点

序号	1	2	3	4	5	6	7	8	9	10
x	0	1	2	3	4	5	6	7	8	9
y	1	1	1	-1	-1	-1	1	1	1	-1
$H_2(x)$	1.07	1.07	1.07	0.23	0.23	0.23	0.23	0.23	0.23	-1.07
$\text{sign}(H_2(x))$	1	1	1	1	1	1	1	1	1	-1



试一试

3. $t=2$ 时, 调整权重, $\mathbf{w}_3 = (w_{31}, w_{32}, \dots, w_{310})$,

$$w_{3j} = \frac{w_{2j} \exp(-\alpha_2 y_i h_2(x_j))}{\sum_{i=1}^m w_{2i} \exp(-\alpha_2 y_i h_2(x_i))}, j = 1, 2, \dots, 10$$

得到:

$$\mathbf{w}_3 = (0.0455, 0.0455, 0.0455, \mathbf{0.1667}, \mathbf{0.1667}, \mathbf{0.1667}, 0.106, 0.106, 0.106, 0.0455),$$

可以看出, 在下一轮的训练中, $x = 3, 4, 5$ 的数据点将更高的权重, 而在本轮训练中被正确识别的样本权重有所降低

序号	1	2	3	4	5	6	7	8	9	10
x	0	1	2	3	4	5	6	7	8	9
y	1	1	1	-1	-1	-1	1	1	1	-1
$H_2(x)$	1.07	1.07	1.07	0.23	0.23	0.23	0.23	0.23	0.23	-1.07
$\text{sign}(H_2(x))$	1	1	1	1	1	1	1	1	1	-1



试一試

4. $t=3$ 时，当前权值分布 w_3 下错误率最低的阈值为5.5，即

$$h_3(x) = \begin{cases} -1, & x < 5.5 \\ 1, & x > 5.5 \end{cases}$$

$h_3(x)$ 的误分类样本为 $x = 0、1、2、9$ 时的样本，误差率为 $\varepsilon_3 = w_{31} + w_{32} + w_{33} +$

$w_{310} = 0.18$ ，计算系数 $\alpha_3 = \frac{1}{2} \log\left(\frac{1}{\varepsilon_3} - 1\right) = 0.7514$

此时分类器 $\text{sign}[H_3(x)]$ 在训练数据集上没有误分类点。

序号	1	2	3	4	5	6	7	8	9	10
x	0	1	2	3	4	5	6	7	8	9
y	1	1	1	-1	-1	-1	1	1	1	-1
$H_3(x)$	0.32	0.32	0.32	-0.53	-0.53	-0.53	0.98	0.98	0.98	-0.32
$\text{sign}(H_3(x))$	1	1	1	-1	-1	-1	1	1	1	-1



试一试

5. 训练结束，强分类器为 H_3 ，即

$$H_3(x) = 0.4236h_1(x) + 0.6496h_2(x) + 0.7514h_3(x)$$

其中，

$$h_1(x) = \begin{cases} 1, & x < 2.5 \\ -1, & x > 2.5 \end{cases}$$

$$h_2(x) = \begin{cases} 1, & x < 8.5 \\ -1, & x > 8.5 \end{cases}$$

$$h_3(x) = \begin{cases} -1, & x < 5.5 \\ 1, & x > 5.5 \end{cases}$$

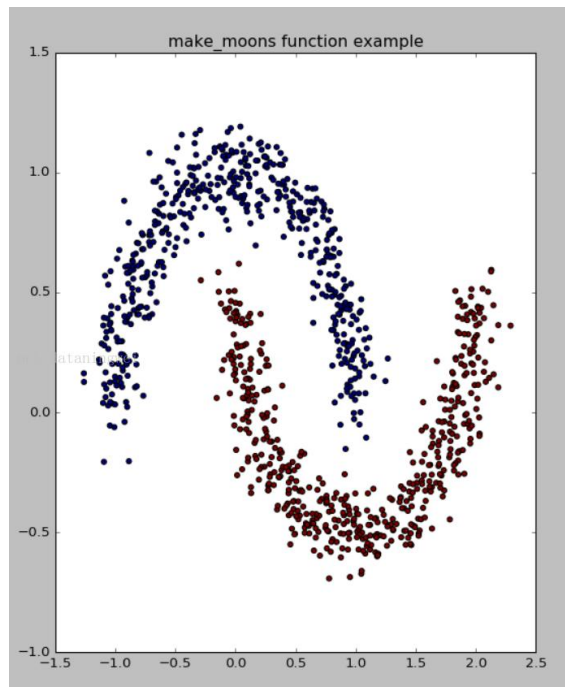
序号	1	2	3	4	5	6	7	8	9	10
x	0	1	2	3	4	5	6	7	8	9
y	1	1	1	-1	-1	-1	1	1	1	-1
$H_3(x)$	0.32	0.32	0.32	-0.53	-0.53	-0.53	0.98	0.98	0.98	-0.32
$\text{sign}(H_3(x))$	1	1	1	-1	-1	-1	1	1	1	-1



试一试

利用sklearn的make_moons数据集（包含两个特征）来实现AdaBoost算法的编码与操作，观察算法效果

Make_moons是sklearn自带的数据集之一，可以生成**环形数据**（生成两个相互交错的半圆），主要用于测试聚类 and 分类算法。该数据集可产生如下所示的数据分布。



图片来源: https://blog.csdn.net/YangWei_19/article/details/79945746



试一试

从make_moons数据集中随机选取部分数据点（隶属于不同半圆的点，标注不同的类别和颜色）

(1) 引入必要的python库，构建AdaBoost类。

首先，定义一个AdaBoostClassifier类，类中包含4个基础参数——基分类器（base_clf）、最大基分类器数量（n_estimators）、收敛精度（tol）、随机状态（random_state）。其中，默认基分类器被设置为深度为1的决策树模型。

```
from sklearn.tree import DecisionTreeClassifier
from copy import deepcopy
import numpy as np
class AdaBoostClassifier():
    def __init__(self, base_clf=None, n_estimators=20, tol=1e-3, random_state=42):
        if base_clf is None:
            base_clf = DecisionTreeClassifier(max_depth=1)
        self.base_clf = base_clf → # 定义基分类器
        self.n_estimators = n_estimators → # 基分类器数量T
        self.tol = tol → # 收敛精度
        self.random_state = random_state → # 随机状态
```



试一试

实现AdaBoost算法，观察算法效果

(2) 构造成员函数

为AdaBoostClassifier类添加一个新的成员函数fit，作为执行AdaBoost算法的入口。此外，也定义了两个类属性fitted_clf和weights，用于在训练结束后获取各个基分类器以及每一轮迭代的样本权重

训练模型

```
def fit(self, X, y):  
    assert X.shape[0] == y.shape[0] —→ # 确保特征与标签长度一致  
    self.fitted_clf = [] —→ # 基分类器列表  
    self.alphas = [] —→ # 基分类器权重  
    self.n = X.shape[0] —→ # 样本数量  
    self.weights = [np.full(self.n, 1 / self.n)] —→ # 样本权重初始化为1/n  
    self._boost(X, y) —→ # 训练基分类器  
    return self
```



试一试

实现AdaBoost算法，观察算法效果

(3) 集成算法训练

算法的核心迭代过程由AdaBoost类的成员函数_boost实现，包括基分类器的训练和权重的更新。权重更新部分的代码被封装在了另一个成员函数_update_weights中

```
def _boost(self, X, y):
    for i in range(self.n_estimators):
        clf = deepcopy(self.base_clf)  # 建立新的基分类器
        clf.set_params(random_state=self.random_state + i)  # 设置基分类器随机状态
        clf.fit(X, y, sample_weight=self.weights[-1])  # 训练基分类器
        y_pred = clf.predict(X)  # 基分类器预测
        error = np.sum(self.weights[-1] * (y_pred != y))  # 计算基分类器训练集错误率
        if error >= 0.5:  # 检查基分类器性能是否优于随机猜测
            break
        self.fitted_clf.append(clf)  # 保存基分类器
        if error < self.tol:  # 检查基分类器是否提前达到预定精度
            self.alphas.append(100)
            break
        alpha = np.log(1 / error - 1) / 2  # 计算基分类器权重
        self.alphas.append(alpha)
        self._update_weights(y_pred, y)  # 更新样本权重
    def _update_weights(self, y_pred, y):
        beta = np.exp(-self.alphas[-1] * y * y_pred)
        self.weights.append(self.weights[-1] * beta)  # 更新样本权重
        self.weights[-1] /= np.sum(self.weights[-1])  # 归一化样本权重
```



试一试

实现AdaBoost算法，观察算法效果

(4) 分类器集成与预测

```
def predict(self, X):  
    h = np.array([clf.predict(X) for clf in self.fitted_clf]) → # 基分类器分别预测  
    ah = np.repeat(self.alphas, X.shape[0], axis=0).reshape(  
        len(self.alphas), -1) * h  
    h_final = np.sign(np.sum(ah, axis=0)) → # 集成预测结果  
    return h_final
```

(5) 导入模拟数据，观察算法效果

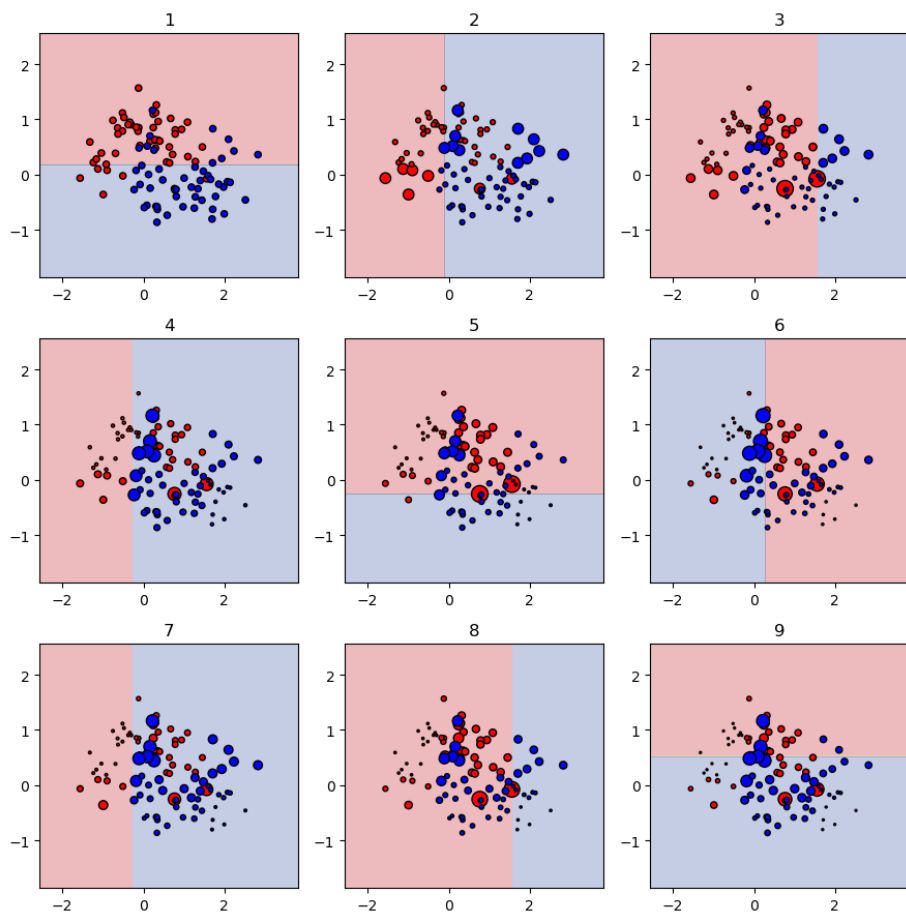
```
from sklearn.datasets import make_moons  
data = make_moons(noise=0.3, random_state=42) → # 生成模拟数据  
X = data[0]  
y = data[1]  
y[y == 0] = -1 → # 修改负样本标签
```

导入
make_
moons
数据集

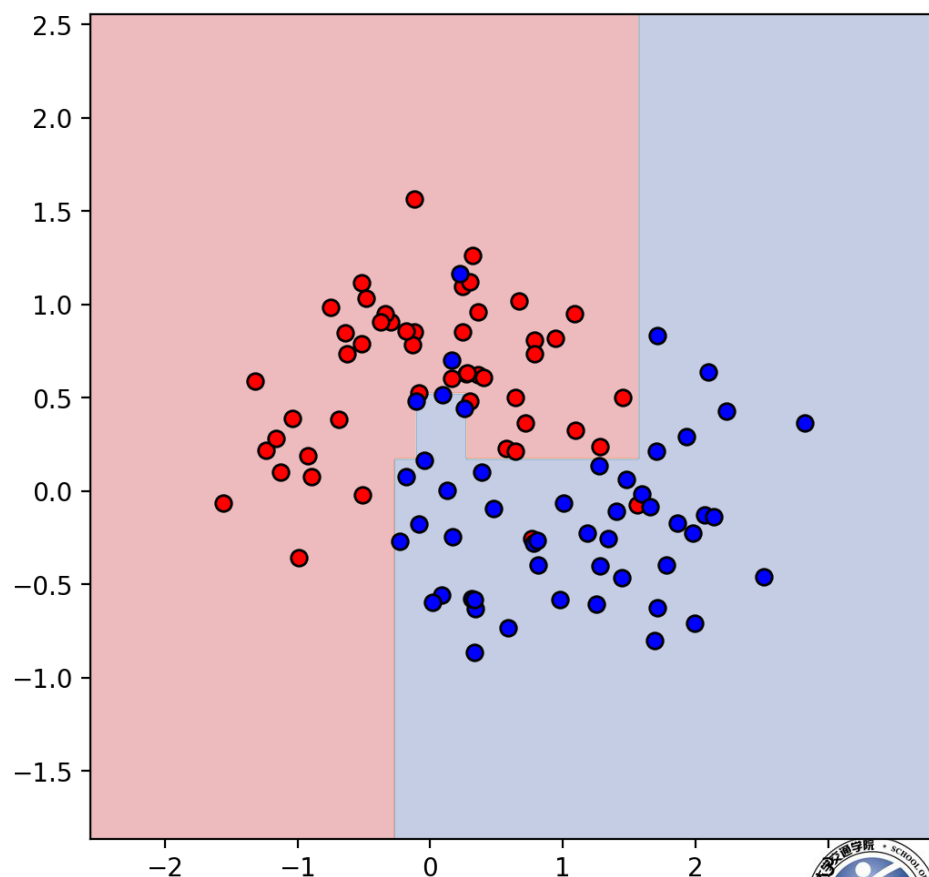


试一试

实现AdaBoost算法，观察算法效果。观察9个基学习器的决策边界与样本权重（样本点的半径大小）



基学习器决策边界



集成模型决策边界



提升树是以决策树为基模型的一类Boosting算法。由于其优异的性能，提升树模型已成为应用最为广泛的机器学习模型之一。

更广义地来讲，提升树实际上属于模型为**加法模型** (Additive Model)，求解算法为**前向分步算法** (Forward Stagewise Algorithm) 的一类学习方法。这里先对加法模型和前向分布算法分别进行介绍。

加法模型

加法模型的数学模型可以表示为下式：

$$f(x) = \sum_{t=1}^T \beta_t h(x, \theta_t)$$

其中， $h(x, \theta_t)$ 为基学习器， θ_t ， β_t 均为训练参数，其中 θ_t 为基学习器模型参数， β_t 为权重参数， t 为基学习器的序号， T 为基学习器总数。

上式表明，集成模型 $f(x)$ 的输出是各个基学习器输出的加权和。



给定损失函数 $L(y, f(x))$ 与训练数据的情况下, 学习上述加法模型的目标是最小化损失函数:

$$\min_{\beta, \theta} \sum_{i=1}^m L\left(y_i, \sum_{t=1}^T \beta_t h(x_i, \theta_t)\right)$$

上式中, m 为训练样本总量。然而, 求解上述优化问题一次性训练 T 个基学习器是非常困难的。**前向分步算法**为求解上述优化问题提供了一个高效可行的方案: **将同时求解所有基学习器参数的优化问题简化为逐次求解各个参数的优化问题。**



前向分步算法

前向分步算法求解加法模型中参数的基本思想是：因为学习的是加法模型，如果能够**从前向后，每一步只学习一个基函数及其系数**，逐步逼近优化目标函数式，那么就可以简化优化的复杂度。具体地，每步（第 t 步）只需优化如下损失函数：

$$\min_{\beta_t, \theta_t} \sum_{i=1}^m L\left(y_i, \sum_{k=1}^{t-1} \beta_k h(x_i, \theta_k) + \beta_t h(x_i, \theta_t)\right) = \min_{\beta_t, \theta_t} \sum_{i=1}^m L(y_i, f_{t-1}(x_i) + \beta_t h(x_i, \theta_t))$$

其中， $f_{t-1}(x_i) = \sum_{k=1}^{t-1} \beta_k h(x_i, \theta_k)$ 表示前 $t-1$ 轮迭代训练得到的基学习器的集成结果。



给定训练数据 $\{(x_1, y_1), \dots, (x_n, y_n)\}$, 损失函数 $L(y, f(x))$, 基函数集 $\{h(x, \theta)\}$, **前向分步算法的流程**如下所示:

(1)初始化 $f_0(x) = 0$

(2)对于 $t = 1, 2, \dots, T$:

(a)极小化损失函数:

$$\min_{\beta_t, \theta_t} \sum_{i=1}^m L(y_i, f_{t-1}(x_i) + \beta_t h(x_i, \theta_t))$$

得到**最优参数** β_t, θ_t

(b)更新:

$$f_t(x) = f_{t-1}(x) + \beta_t h(x, \theta_t)$$

(3)得到加法模型:

$$f(x) = \sum_{t=1}^T \beta_t h(x, \theta_t)$$



提升树

将上述加法模型中的**基学习器**替换为**决策树模型**（对于分类问题而言是分类树，回归问题是回归树），就可以得到**提升树模型**。记一棵输入为 x 的决策树模型输出结果为 $h(x, \theta_t)$ ，其中 θ_t 为决策树 t 的参数，考虑到模型复杂度和过拟合问题， β_t 在提升树中被固定为超参数。 T 棵决策树的最终集成结果可以写为：

$$H_T(x) = \sum_{t=1}^T \beta_t h(x, \theta_t)$$

在提升树算法每一轮迭代中，新一轮的预测结果由**前一轮的集成模型预测结果**与**新构造的决策树预测结果**相加得到。第 t 轮的预测结果为：

$$H_t(x) = H_{t-1}(x) + \beta_t h(x, \theta_t)$$



提升树

在训练提升树时，采用前向分步算法依次优化决策树参数 θ_t ，以使模型损失最小化，记损失函数为 $L(\cdot, \cdot)$ ，则第 t 轮训练得到决策树最优参数 θ_t^* 通过如下计算得到：

$$\theta_t^* = \underset{\theta_t}{\operatorname{argmin}} L(\mathbf{y}, H_{t-1}(\mathbf{x}) + \beta_t h(\mathbf{x}, \theta_t))$$

在处理不同问题（分类/回归）时，通常会选择不同的**损失函数**。例如，在**回归问题上**常采用**均方误差损失函数**，而**分类问题**常采用**指数损失函数**。在提升树的训练中，需要根据不同的损失函数会设计相应的求解算法。本节课将针对上述情况依次进行介绍，最终介绍处理一般化损失函数的梯度提升方法

$$\mathcal{L}^{MSE}(\mathbf{y}, \hat{\mathbf{y}}) = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2$$

均方误差损失函数

$$\mathcal{L}^{exp}(\mathbf{y}, \hat{\mathbf{y}}) = \frac{1}{m} \sum_{i=1}^m \exp(-y_i \hat{y}_i)$$

指数损失函数



提升回归树

在**回归问题**中，提升树通常使用均方误差（MSE）作为损失函数：

$$\mathcal{L}_{MSE}(\mathbf{y}, \hat{\mathbf{y}}) = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2$$

其中 y_i 为真实特征， $\hat{y}_i$ 为对应的预测结果。在这个算法 $h(\mathbf{x}, \boldsymbol{\theta}_t)$ 需尽量去逼近残差， β_t 直接取1，因此，提升回归树在第 t 轮迭代的损失函数可以写作：

$$\mathcal{L}_{MSE}(\mathbf{y}, H_t(\mathbf{x})) = \frac{1}{m} \sum_{i=1}^m (y_i - H_t(\mathbf{x}_i))^2$$

$$= \frac{1}{m} \sum_{i=1}^m [y_i - (H_{t-1}(\mathbf{x}_i) + h(\mathbf{x}_i; \boldsymbol{\theta}_t))]^2$$

$$= \frac{1}{m} \sum_{i=1}^m [y_i - H_{t-1}(\mathbf{x}_i) - h(\mathbf{x}_i; \boldsymbol{\theta}_t)]^2$$

⇓ 当前模型的残差

$$= \frac{1}{m} \sum_{i=1}^m [\varepsilon_{t,i} - h(\mathbf{x}_i; \boldsymbol{\theta}_t)]^2$$

$$= \mathcal{L}_{MSE}(\boldsymbol{\varepsilon}_t, h(\mathbf{x}; \boldsymbol{\theta}_t))$$

特征不变，以当前模型的残差 $(y_i - H_{t-1}(\mathbf{x}_i))$ 为标签训练基模型！

以残差为标签（代替了 y_i ）
训练决策树（损失函数仍为MSE）



提升回归树

提升回归树的具体算法步骤可总结如下：

(1) 初始化回归树

$$H_0(\mathbf{x}) = 0$$

(2) 对决策树 t , $t \in \{1, \dots, T\}$ 进行以下操作：

① 对每个样本 $i = 1, 2, \dots, m$, 计算前一轮迭代中模型残差：

$$\varepsilon_{t,i} = y_i - H_{t-1}(\mathbf{x}_i), \quad i = 1, 2, \dots, m$$

② 以残差 ε_t 为标签，输入特征不变，训练回归树模型 $h(\mathbf{x}, \boldsymbol{\theta}_t)$ ，并计算本轮预测结果：

$$H_t(\mathbf{x}) = H_{t-1}(\mathbf{x}) + \beta_t h(\mathbf{x}, \boldsymbol{\theta}_t)$$

其中 $\beta_t = 1$.

(3) 最终的提升树由各轮构造的决策树相加得到

$$H_T(\mathbf{x}) = \sum_{i=1}^T \beta_t h(\mathbf{x}, \boldsymbol{\theta}_t)$$



提升分类树

在**分类问题**中，提升树通常使用指数损失函数或交叉熵损失函数。在**以指数损失函数为损失函数处理二分类问题**时，构造得到的**提升分类树**和前面介绍的**AdaBoost**是等价的。感兴趣的同学可以自行阅读李航所著的《统计学习方法》第八章的内容查看证明。

指数损失函数:

$$\mathcal{L}^{exp}(\mathbf{y}, \hat{\mathbf{y}}) = \frac{1}{m} \sum_{i=1}^m \exp(-y_i \hat{y}_i)$$

交叉熵损失函数:

$$\mathcal{L}^{entropy}(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_{k=1}^{|\mathbf{y}|} p_k \log_2(q_k)$$

其中，输出空间 $\mathbf{y}$ 的可能取值包括 $k = 1, 2, 3, \dots, |\mathbf{y}|$ ，第 k 类样本占 $\mathbf{y}$ 比例为 p_k ， $\hat{\mathbf{y}}$ 中第 k 类样本的比例为 q_k



梯度提升树 (Gradient Boosting Tree)

提升树利用加法模型与前向分步算法实现学习的优化过程，当损失函数是平方损失和指数损失函数时，每一步优化是很简单的。但**对一般损失函数而言，往往每一步优化并不那么容易**。针对这一问题，Freidman 提出了梯度提升 (Gradient Boosting) 算法。这是利用最速下降法的近似方法，**关键是利用损失函数的负梯度在当前模型的值**。

计算损失函数负梯度

考虑一般损失函数情况下，第 $t - 1$ 轮训练结束后的损失：

$$L(\mathbf{y}, H_{t-1}(\mathbf{x}))$$

根据加法模型的构造，在第 t 轮训练后，我们希望得到 $H_t(\mathbf{x}) = H_{t-1}(\mathbf{x}) + \beta_t h(\mathbf{x}, \boldsymbol{\theta}_t)$ ，使 $L(\mathbf{y}, H_t(\mathbf{x}))$ 相比于 $L(\mathbf{y}, H_{t-1}(\mathbf{x}))$ 得到最大程度的下降。因此，如果**把 $H_{t-1}(\mathbf{x})$ 当作损失函数的变量**，那么使损失函数下降最快的下降方向可以写作：

$$-\nabla_{H_{t-1}(\mathbf{x})} \mathcal{L}(\mathbf{y}, H_{t-1}(\mathbf{x})) = -\frac{\partial L(\mathbf{y}, H_{t-1}(\mathbf{x}))}{\partial H_{t-1}(\mathbf{x})}$$



梯度提升树

给定损失函数下降最快的方向：

$$-\nabla_{H_{t-1}(x)} \mathcal{L}(y, H_{t-1}(x)) = -\frac{\partial L(y, H_{t-1}(x))}{\partial H_{t-1}(x)}$$

那么**第t轮迭代之后的最小损失函数**可以写作：

$$L\left(y, H_{t-1}(x) - \beta_t \frac{\partial L(y, H_{t-1}(x))}{\partial H_{t-1}(x)}\right)$$

其中， β_t 取0-1范围内的值，注意和回归提升树（ $\beta_t = 1$ ）区分。提升树中为了尽量逼近残差， β_t 取1。而梯度提升是借鉴梯度下降的思想， β_t 是步长，所以取0-1范围。

而在提升树模型中，**第t轮迭代之后模型的损失函数**为：

$$L(y, H_t(x)) = L(y, H_{t-1}(x) + \beta_t h(x, \theta_t))$$

用第t轮迭代的基学习器近似第t-1轮损失函数的负梯度，可以近似获得使损失函数下降的最快方向

为了使第t轮训练后的损失函数最小，我们只需训练 $h(x, \theta_t)$ ，使之与 $-\frac{\partial L(y, H_{t-1}(x))}{\partial H_{t-1}(x)}$ 的偏差最小。也就是：**特征不变，以 $-\frac{\partial L(y, H_{t-1}(x))}{\partial H_{t-1}(x)}$ 为标签训练基学习器**



梯度提升树

下面给出梯度提升树的训练步骤：

(1) 初始化回归/分类树

对于回归树，一般取 $H_0(\mathbf{x}) = 0$ ；

对于分类树，若使用交叉熵损失函数 $\mathcal{L}^{entropy}(\mathbf{y}, \hat{\mathbf{y}})$ ，由于涉及到分类概率，为简化梯度的计算，一般假设样本分类到各个类别的概率相同。

(2) 对决策树 t , $t \in \{1, \dots, T\}$ 进行以下操作：

① 计算损失函数对第 $t - 1$ 轮预测结果的负梯度： $-\nabla_{H_{t-1}(\mathbf{x})} \mathcal{L}(\mathbf{y}, H_{t-1}(\mathbf{x}))$

② 以上述负梯度为标签，输入特征不变，训练树模型 $h(\mathbf{x}, \boldsymbol{\theta}_t)$ ，并计算本轮预测结果：

$$H_t(\mathbf{x}) = H_{t-1}(\mathbf{x}) + \beta_t h(\mathbf{x}, \boldsymbol{\theta}_t)$$

(3) 最终的提升树由各轮构造的决策树相加得到

$$H_T(\mathbf{x}) = \sum_{i=1}^T \beta_i h(\mathbf{x}, \boldsymbol{\theta}_i)$$



试一试

给定如下图所示的训练数据， x 的取值范围为 $[0.5, 10.5]$ ， y 的取值范围为 $[5, 10]$ ，基模型为只有一个根节点和两个叶节点的回归树，学习提升回归树模型（系数 β_t 设置为1）

x_i	1	2	3	4	5	6	7	8	9	10
y_i	5.56	5.70	5.91	6.40	6.80	7.05	8.90	8.70	9.00	9.05

解：

第一步：构造回归树 $h_1(x)$ ：

回归树需要求解以下优化问题得到：

$$\min_s \left\{ \sum_{x_i \in R1} (y_i - c1)^2 + \sum_{x_i \in R2} (y_i - c2)^2 \right\}$$

其中， $R1$ 和 $R2$ 分别表示两个叶节点包含的样本数， $R1 = \{x | x < s\}$ ， $R2 = \{x | x \geq s\}$ ， s 为两个叶节点的切分点。 $c1$ 和 $c2$ 分别为两个叶节点的标签



试一试

第一步：构造回归树 $h_1(x)$:

回归树需要求解以下优化问题得到:

$$\min_s \left\{ \sum_{x_i \in R1} (y_i - c1)^2 + \sum_{x_i \in R2} (y_i - c2)^2 \right\}$$

$c1$ 和 $c2$ 分别为两个叶节点的标签:

$$c1 = \frac{\sum_{x_i \in R1} y_i}{N1}, \quad c2 = \frac{\sum_{x_i \in R2} y_i}{N2}$$

其中, $N1$ 和 $N2$ 分别为 $R1$ 和 $R2$ 中样本的数目。



试一试

第一步：构造回归树 $h_1(x)$:

接下来讨论计算数据切分点的计算，观察到样本数据的特征，我们考虑以下切分点：

1.5, 2.5, 3.5, 4.5, 5.5, 6.5, 7.5, 8.5, 9.5。对于每个切分点，可以计算训练误差。

例如取切分点为1.5, $R1 = \{1\}, R2 = \{2, 3 \dots, 10\}$, 此时, $c1 = 5.56, c2 = 7.50$ 。

$$L(s = 1.5) = \sum_{x_i \in R1} (y_i - c1)^2 + \sum_{x_i \in R2} (y_i - c2)^2 = 15.72$$

对于每一个切分点，我们都可以计算得到训练误差：

s	1.5	2.5	3.5	4.5	5.5	6.5	7.5	8.5	9.5
$L(s)$	15.72	12.07	8.36	5.78	3.91	1.93	8.01	11.73	15.74

可以看出，切分点取 $s = 6.5$ 时，训练误差最小。此时， $c1 = 6.24, c2 = 8.91$



试一试

第一步：构造回归树 $h_1(x)$:

我们可以得到回归树 $h_1(x)$:

$$h_1(x) = \begin{cases} 6.24 & x < 6.5 \\ 8.91 & x \geq 6.5 \end{cases}$$

此时提升树为:

$$H_1(x) = h_1(x) = \begin{cases} 6.24 & x < 6.5 \\ 8.91 & x \geq 6.5 \end{cases}$$

用 $H_1(x)$ 拟合数据的残差 ($y_i - H_{t-1}(x_i)$) 如下表所示

x_i	1	2	3	4	5	6	7	8	9	10
r_{2i}	-0.68	-0.54	-0.33	0.16	0.56	0.81	-0.01	-0.21	0.09	0.14

此时的训练误差为:

$$\sum_{i=1}^{10} (y_i - H_1(x_i))^2 = 1.93$$



试一试

第二步：构造回归树 $h_2(x)$

重复上述步骤，训练回归树 $h_2(x)$ 。与上述步骤不同之处在于**训练数据的标签用上表中的残差**。

x_i	1	2	3	4	5	6	7	8	9	10
ε_{2i}	-0.68	-0.54	-0.33	0.16	0.56	0.81	-0.01	-0.21	0.09	0.14

重复上述步骤可以得到回归树 $h_2(x)$ ：

$$h_2(x) = \begin{cases} -0.52 & x < 3.5 \\ 0.22 & x \geq 3.5 \end{cases}$$

此时提升树为：

$$H_2(x) = h_1(x) + h_2(x) = \begin{cases} 5.72 & x < 3.5 \\ 6.46 & 3.5 \leq x < 6.5 \\ 9.13 & x \geq 6.5 \end{cases}$$



试一试

第二步：构造回归树 $h_2(x)$

此时提升树的训练误差为：

$$\sum_{i=1}^{10} (y_i - H_2(x_i))^2 = 0.79$$

可以看出，通过集成两个基模型，训练误差能够显著减小

重复上述步骤，**以残差为目标函数训练回归树模型**，可以得到各个轮次的回归树模型



试一试

重复上述步骤，以残差为目标函数训练回归树模型，可以得到各个轮次的回归树模型：

$$h_3(x) = \begin{cases} 0.15 & x < 6.5 \\ -0.22 & x \geq 6.5 \end{cases}$$

$$\sum_{i=1}^{10} (y_i - H_3(x_i))^2 = 0.47$$

$$h_4(x) = \begin{cases} -0.16 & x < 4.5 \\ 0.11 & x \geq 4.5 \end{cases}$$

$$\sum_{i=1}^{10} (y_i - H_4(x_i))^2 = 0.30$$

$$h_5(x) = \begin{cases} 0.07 & x < 6.5 \\ -0.11 & x \geq 6.5 \end{cases}$$

$$\sum_{i=1}^{10} (y_i - H_5(x_i))^2 = 0.23$$

$$h_6(x) = \begin{cases} -0.15 & x < 2.5 \\ 0.04 & x \geq 2.5 \end{cases}$$

$$\sum_{i=1}^{10} (y_i - H_6(x_i))^2 = 0.17$$



试一试

最终得到的集成模型是：

$$H_6(x) = h_1(x) + h_2(x) + \cdots + h_6(x) = \begin{cases} 5.63 & x < 2.5 \\ 5.82 & 2.5 \leq x < 3.5 \\ 6.56 & 3.5 \leq x < 4.5 \\ 6.83 & 4.5 \leq x < 6.5 \\ 8.95 & x \geq 6.5 \end{cases}$$

$$h_1(x) = \begin{cases} 6.24 & x < 6.5 \\ 8.91 & x \geq 6.5 \end{cases}$$

$$h_2(x) = \begin{cases} -0.52 & x < 3.5 \\ 0.22 & x \geq 3.5 \end{cases}$$

$$h_3(x) = \begin{cases} 0.15 & x < 6.5 \\ -0.22 & x \geq 6.5 \end{cases}$$

$$h_4(x) = \begin{cases} -0.16 & x < 4.5 \\ 0.11 & x \geq 4.5 \end{cases}$$

$$h_5(x) = \begin{cases} 0.07 & x < 6.5 \\ -0.11 & x \geq 6.5 \end{cases}$$

$$h_6(x) = \begin{cases} -0.15 & x < 2.5 \\ 0.04 & x \geq 2.5 \end{cases}$$



以 $x < 2.5$ 时模型输出为例，在 $x < 2.5$ 时，模型预测结果为各个基模型在 $x < 2.5$ 情况下输出的加和：

$$\begin{aligned} & 6.24 - 0.52 + 0.15 \\ & - 0.16 + 0.07 - 0.15 \\ & = 5.63 \end{aligned}$$



3 Bagging 算法

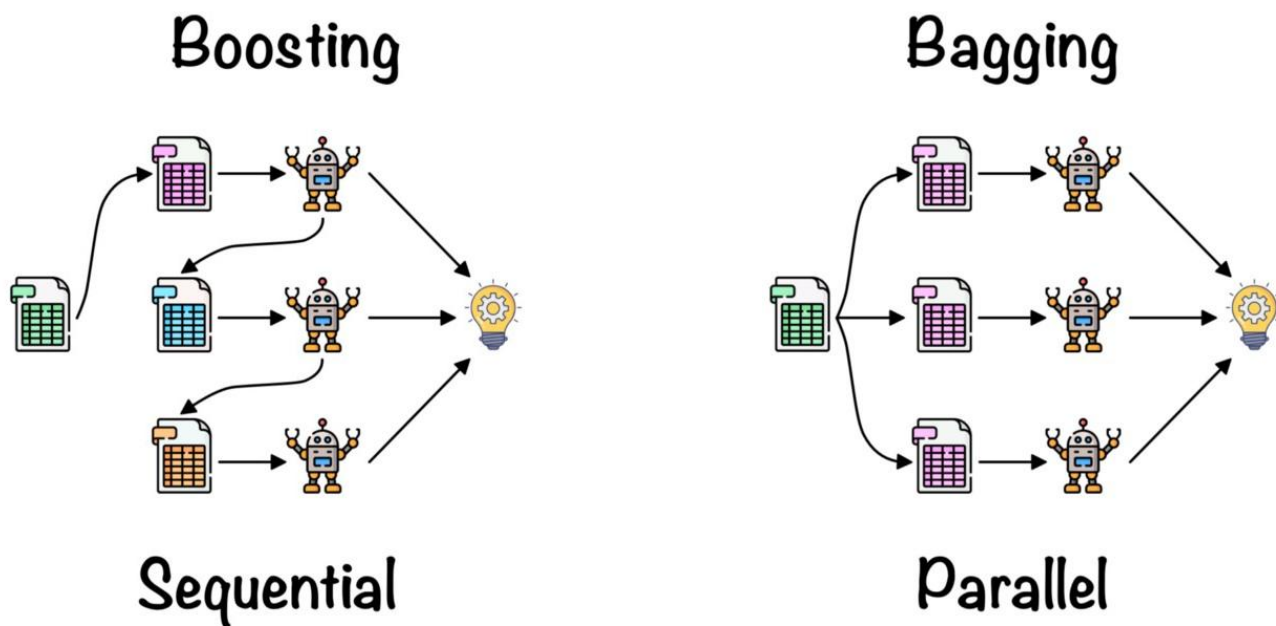
- Bagging 简介

- 随机森林



Bagging 简介

大家可以注意到，上述介绍的几类Boosting算法都是采用串行的方式进行训练。不同于Boosting算法，接下来我们将要介绍的**Bagging算法采用并行的训练方式，各个基学习器之间相互独立。Bagging算法通过构造独立而多样的学习器增强模型的泛化能力**



Bagging算法通过多个学习器共同表决投票得出集成结果

Bagging 简介

请思考：

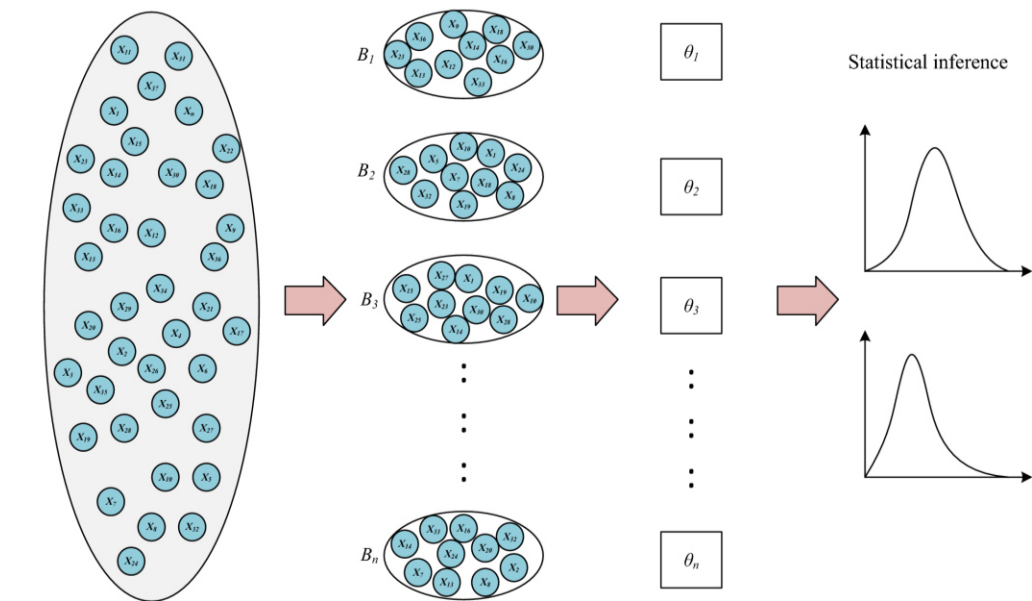
- (1) Bagging算法主要降低模型的偏差还是方差？为什么
- (2) 从计算效率上来看，Bagging算法具有什么优势？
- (3) 给定数据集，如何设计训练策略才能构造独立而多样的学习器以增强模型的泛化能力？



Bagging 简介

为了在有限的数据条件下，为各个个体学习器构造尽可能独立且存在一定差异的训练集，一种常用的解决方法就是**对训练样本进行重新采样，产生出若干个不同的子集。基于这些数据子集，可以分别训练出若干个体学习器，最终将它们集成。**

为了保证个体学习器的性能，需要保证训练各个个体学习器的数据集子集包含了充分的信息。因此在采样时，尤其对于样本数量不大的数据集，**一般会让采样出的数据集子集互斥，而是使它们相互有重叠。**



Bagging 简介

有放回采样（Bootstrap）是Bagging算法中常用的一种重采样方法。**对于一个有 m 个样本的数据集，有放回地选取 m 个数据，组成的新数据集显然也包含 m 个样本数据。**在有放回采样生成的数据集中，原始数据集中的一些数据会重复出现，而一些数据则未被包含在新数据集中。当 m 充分大时，一个数据不在数据集中的概率收敛于：

$$\lim_{m \rightarrow \infty} \left(1 - \frac{1}{m}\right)^m = \frac{1}{e} = 36.8\%$$

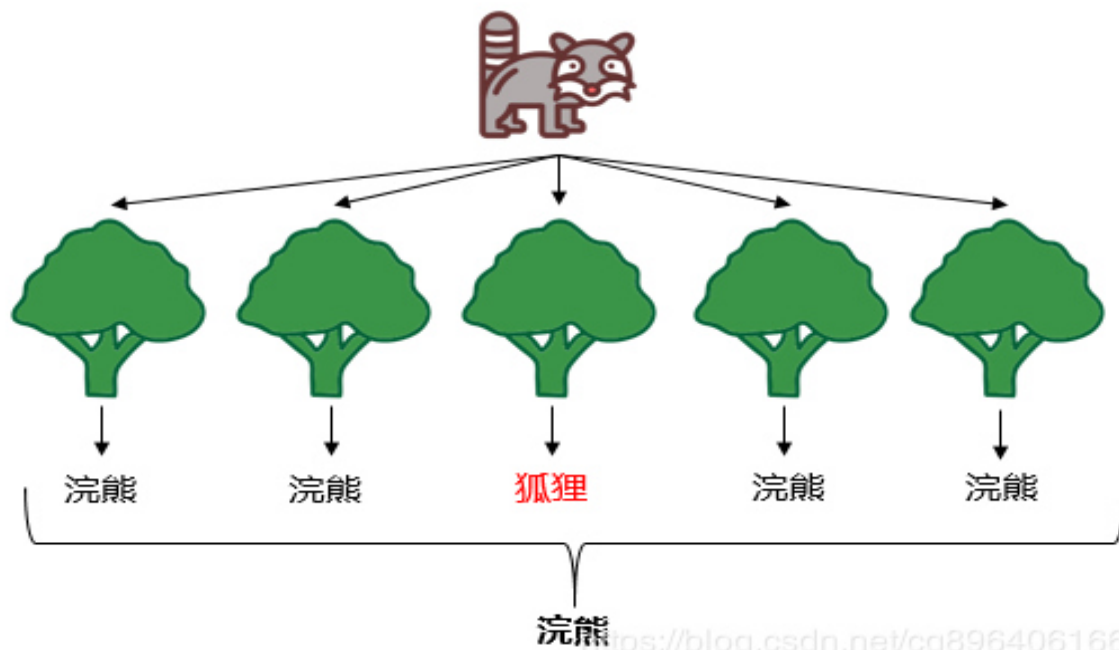
仅有约2/3的样本被用来训练一个个体学习器。对于剩余的约1/3的样本，它们会被作为验证集，来评估该个体学习器的泛化性能。评估结果通常被称为**包外（out-of-bag）估计**。包外估计在模型训练中还可以帮助减小模型的过拟合风险。例如，若基学习器是神经网络，包外估计的结果可以用作早停策略执行的依据；若基学习器是决策树，它也可以作为模型的剪枝的参考。



随机森林 (Random Forest, RF)

随机森林是基于决策树模型设计的一种Bagging集成模型。有学者将随机森林定义为一个由一系列决策树组成的分类器。

在随机森林模型中，**基于所有决策树的投票，随机森林将给出最终的集成结果**。对于回归任务，只需要将随机森林的基学习器由分类树替换为回归树，并将最终的投票法替换为简单平均法即可。



随机森林

在Bagging集成方法中，各个基学习器会在重采样的数据集上进行训练，随后对结果投票来提升预测精度。通过这种方式，随机森林可以建立数十或数百棵树，形成一片决策“森林”。为了丰富森林中决策树的多样性，**除了Bagging中的数据集重采样，许多研究曾尝试为这些基学习器引入更多的随机性**，这些尝试包括：

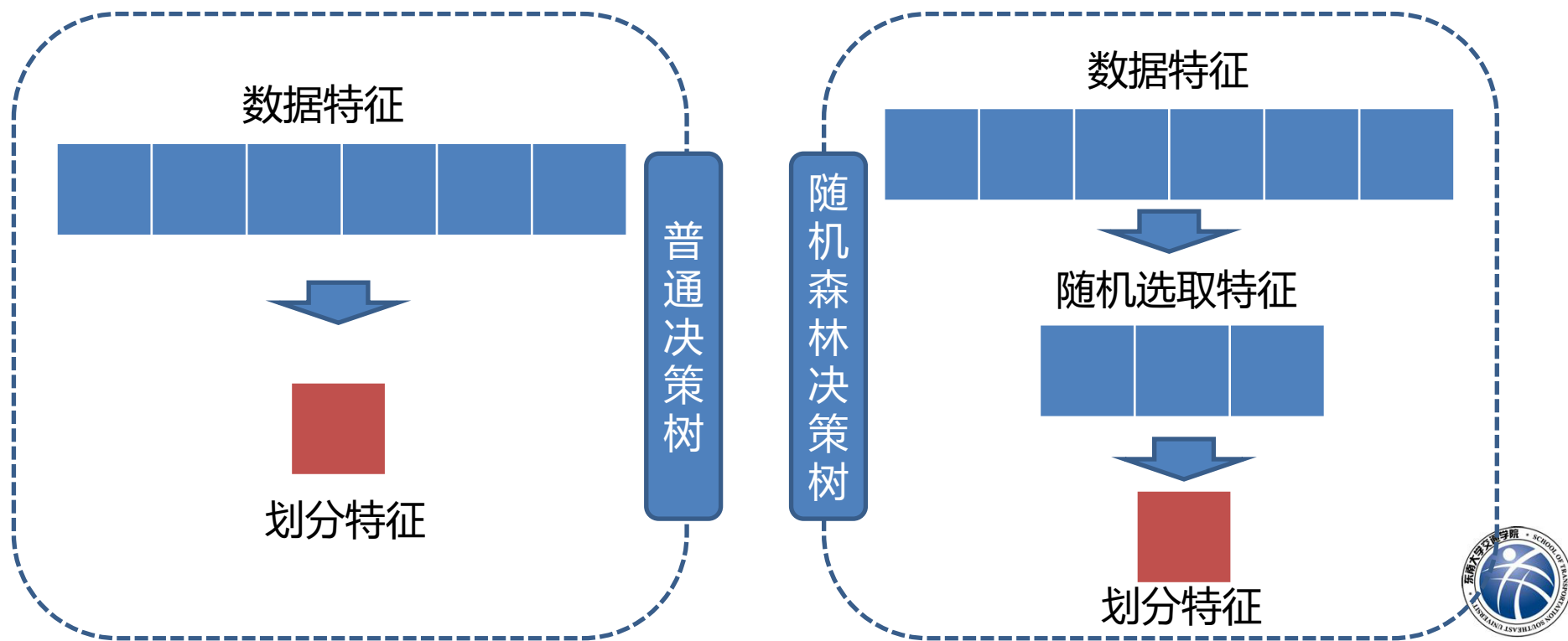
- 引入随机特征选择机制
- 在决策树的输出中添加随机噪声
- 在决策树分支划分时，从多个划分特征中随机选取划分点
- ...

其中，以**随机特征选择**机制效果最为显著，现在所广泛使用的随机森林模型在bagging集成方法的基础上加入了随机特征选择机制



随机森林

普通决策树在划分子节点时，通常会考虑全部的 d 维输入特征，而随机森林中的决策树则会对特征再进行一次采样，即从全部特征中随机选择 k 个，再在这 k 维特征中寻找最优的划分点。这里的超参数 k 控制了基决策树的随机程度，特别地，当 $k = d$ 时，随机森林等价于基学习器为普通决策树的Bagging集成模型。在实践中推荐取值为 $k = \log_2 d$



随机森林

给定数据集 $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$, 随机森林的**算法步骤**可以总结如下:

(1) 对于决策树 $t = 1, 2, \dots, T$ (T 为随机森林决策树的个数):

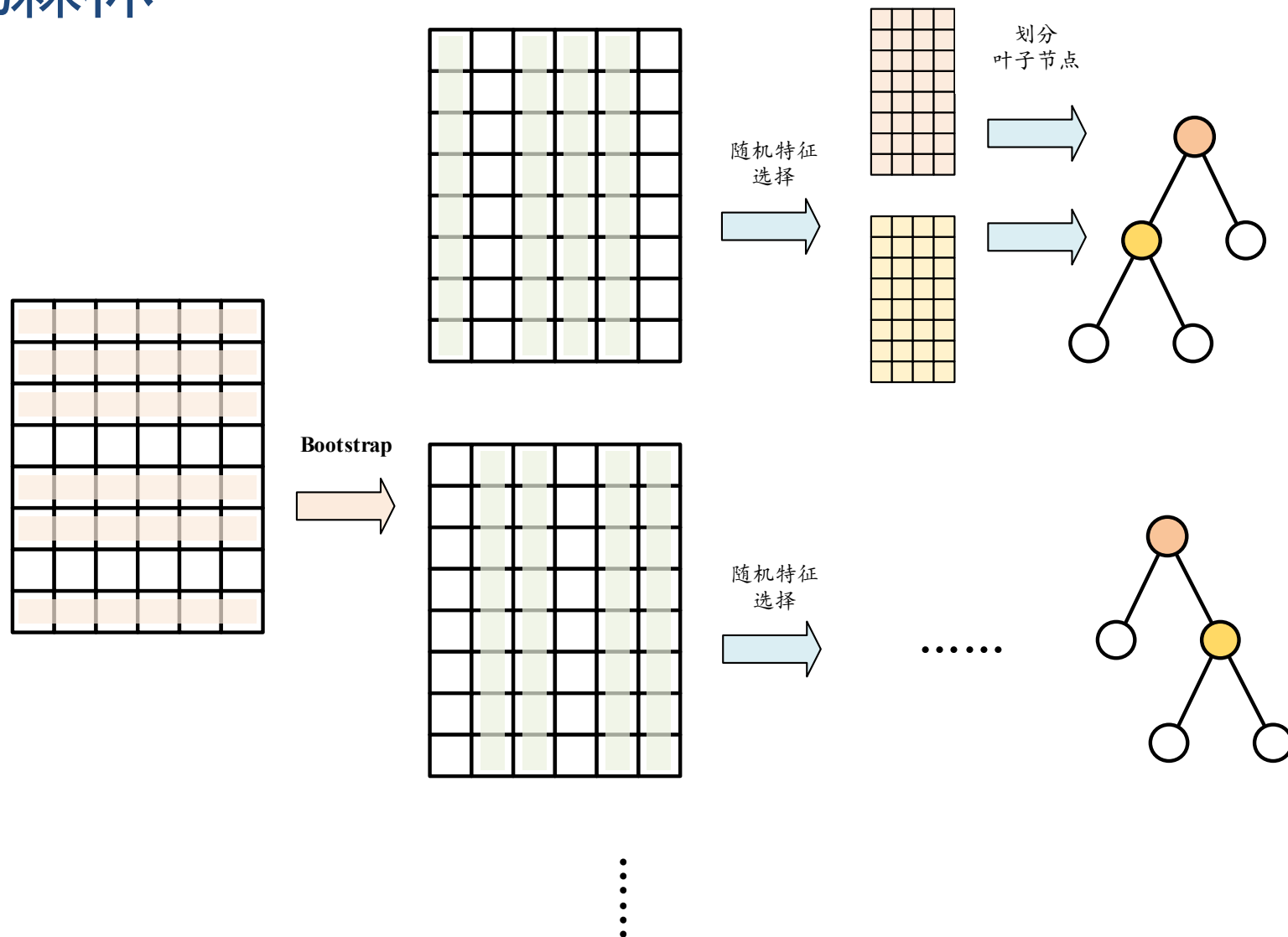
- ① 对原始数据进行有放回采样 (Bootsrtap) , 得到新训练集 D_b , D_b 将用于训练决策树 t
- ② 在 (父节点对应的) 训练集的所有特征中随机选择 k 个特征
- ③ 遍历 k 个特征, 寻找决策树的**最优划分特征与划分点**, 划分样本 (建立子节点)
- ④ 对于每个新生成的节点重复步骤 ②-③, 直到满足决策树的停止规则 (最大深度、误差阈值)

(2) 基于 T 棵决策树的预测结果进行投票, 最终投票结果即为最终集成结果

需要注意的是, 在随机森林算法中通常无需对决策树进行剪枝。但在实践中, 若数据集较大, 可以考虑设置一定的剪枝条件, 以提升计算效率、减小内存消耗



随机森林



随机森林算法示意图

随机森林

通过**Bootstrap采样**和**随机特征选择**，**随机森林从训练样本和输入特征两个方面为模型引入了随机因素**，极大地丰富了基学习器的多样性。尽管随机森林算法的实现非常简单，但这些操作使随机森林模型不易受异常值等噪声数据的干扰，使最终集成模型的泛化性能得到很大的增强。随着基决策树数量的增加，集成模型的泛化误差通常也能进一步降低。

此外，**随机特征选择也提升了随机森林的算法效率**。相比于普通决策树模型，随机森林中的决策树在划分子节点时，不再需要考察全部的特征，仅需要关注采样的特征子集即可。



随机森林

优点:

- 相对而言, 训练速度快。
- 能够处理很高维度的数据, 并且不用做特征选择。
- 在训练过程中, 能够度量出特征间的互相影响程度。
- 如果有很大一部分的特征遗失, 仍可以维持准确度。

缺点:

- 在某些噪音较大的分类或回归问题上会过拟合。



Bagging, Boosting二者的适用范围

1) 样本选择:

Bagging: 训练集是在原始集中有放回选取的, 从原始集中选出的各轮训练集之间是**独立**的;

Boosting: 每一轮的训练集**不变**, 只是训练集中每个样例在分类器中的权重发生变化。而权值是根据上一轮的分类结果进行调整。

2) 样例权重:

Bagging: 使用均匀取样, 每个样例的权重相等;

Boosting: 根据错误率不断调整样例的权值, 错误率越大则权重越大。



Bagging, Boosting二者的适用范围

3) 预测函数:

Bagging: 所有预测函数的权重**相等**。

Boosting: 每个弱分类器都有相应的权重, 对于分类误差小的分类器会有更大的权重。

4) 并行计算:

Bagging: 各个预测函数**可以并行**生成

Boosting: 各个预测函数**只能顺序**生成, 因为后一个模型参数需要前一轮模型的结果



4 个体学习器的多样性

集成学习的强大学习能力来源于对大量的个体学习器的组合。但是，**集成学习对个体学习器的集成是有条件的**，需要注意，**并非任意个体学习器的组合都能带来性能的巨大提升**

- 若个体学习器的**性能极差**，例如为各个样本随机分配标签，则不论使用什么样的集成学习算法，最终结果也不会有任何改善。

	模型 A	模型 B	模型 C	集成结果	真实路况
双龙大道	拥堵	畅行	畅行	畅行	拥堵
中山东路	畅行	畅行	拥堵	畅行	拥堵
龙蟠中路	畅行	拥堵	拥堵	拥堵	拥堵



4 个体学习器的多样性

- 若**个体学习器预测结果高度相似**，则集成结果的意义也不大

	模型 A	模型 B	模型 C	集成结果	真实路况
双龙大道	拥堵	拥堵	拥堵	拥堵	拥堵
中山东路	畅行	畅行	畅行	畅行	拥堵
龙蟠中路	拥堵	拥堵	拥堵	拥堵	拥堵

- 要想建立一个理想的集成学习器，必须保证**个体学习器具有一定的预测能力**，同时，**个体学习器之间具有一定的差异性**

	模型 A	模型 B	模型 C	集成结果	真实路况
双龙大道	拥堵	畅行	拥堵	拥堵	拥堵
中山东路	畅行	拥堵	拥堵	拥堵	拥堵
龙蟠中路	拥堵	拥堵	畅行	拥堵	拥堵



4 个体学习器的多样性

在大多数情况下，使用任意较为成熟的机器学习算法，都能建立具有一定预测能力的模型。因此，对于集成学习中的个体学习器，通常需要关注的是如何避免**个体学习器预测结果高度相似**的情形，即使**学习器具有一定的多样性**。一种最简单的想法是使用不同类型的机器学习算法，也就是前面提到的**异质集成**，另一种方法则是**为模型引入随机干扰**。为模型引入随机干扰的思路主要有四种：

(1) **模型超参数随机**。机器学习模型中往往有许多超参数需要人为指定，例如线性回归的正则项系数、决策树的深度、SVM的核函数等。不同的参数设置有可能对模型的输出结果造成一定的影响。



4 个体学习器的多样性

(2) **训练样本随机**。Bagging集成中的Bootstrap采样就属于这一类方法。在使用时需要注意，**不同机器学习模型对于数据扰动的敏感度不同**，决策树和神经网络等模型易受数据干扰，线性回归和SVM等模型则较为稳定，增加训练样本随机性的方法不适用于稳定的基学习器。

(3) **输入特征随机**。输入特征从多个方面反映样本的属性和特点，是机器学习算法执行分类或回归任务的依据。考虑不同的样本属性，自然可能得到不同的结果，随机森林中提到的随机特征选择方法，正是通过这种方式来增加基学习器的随机性。



4 个体学习器的多样性

(4) **训练标签随机**。通过对模型的训练标签进行修改也可以达到引入随机性的目的，例如改变少量训练样本的标签、为标签添加噪声等。

上述方法通过引入随机性因素，都可以增加基学习器之间的差异，提高基学习器的多样性。另外，**这些方法可以相互组合使用**，例如随机森林就同时向训练样本和输入特征引入了随机干扰，大幅提升了模型的泛化性能。



试一试

利用基于网约车历史轨迹路况数据提取出的网格数据，建立一个用于网格拥堵程度判别的模型，并对比基于随机森林和梯度提升决策树的分类结果

(1) 数据读取，数据预处理，特征选取与数据集划分

```
import pandas as pd
# 读取数据文件
data = pd.read_csv("DATASET-C-B.csv")
# 转换数据类型
for c in ['rowid', 'colid', 'time_id']:
    data[c] = data[c].astype(int)
data = data.sort_values(['date', 'rowid', 'colid', 'time_id']).reset_index(drop=True)
data['datetime'] = pd.to_datetime(data.date, format='%Y%m%d') #转换日期格式
data['dayofweek'] = data.datetime.dt.dayofweek #提取星期信息
data = data.sample(50000, random_state=233) # 随机选取50000条数据

from sklearn.model_selection import train_test_split
features = [
    'rowid', 'colid', 'time_id', 'dayofweek',
    'aveSpeed', 'gridAcc', 'volume', 'speed_std', 'stopNum'
]
train, test = train_test_split(data, test_size=0.3, random_state=233) #数据集划分
```



试一试

(2) 构建随机森林和梯度提升树模型（可自行调整模型参数）

```
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
rf = RandomForestClassifier(
    n_estimators=256,          #森林中的决策树数量
    max_depth=9,              #决策树的最大深度
    min_samples_leaf=30,      #叶子节点包含的最小样本数量
    n_jobs=-1,                #模型拟合使用的处理器数量
    random_state=233          #随机种子，用于获得可复制的结果
) #随机森林模型

gbdt = GradientBoostingClassifier(
    learning_rate=0.05,       #选择模型类型
    n_estimators=256,         #迭代轮数
    max_depth=8,              #决策树的最大深度
    subsample=0.8,            #数据采样比例
    max_features=0.9,         #特征采样比例
    min_samples_split=5,      #划分叶子节点所需的最小样本数量
    min_samples_leaf=30,      #叶子节点包含的最小样本数量
    random_state=233          #随机种子，用于获得可复制的结果
) #梯度提升树模型
```



试一试

(3) 进行模型训练

```
'''随机森林训练'''
rf.fit(train[features], train['labels'])
rf_train_score = rf.score(train[features], train['labels']) #计算训练集精度
rf_test_score = rf.score(test[features], test['labels']) #计算测试集精度
print(f'随机森林模型精度: 训练集: {rf_train_score:.3f}; 测试集: {rf_test_score:.3f}')
```

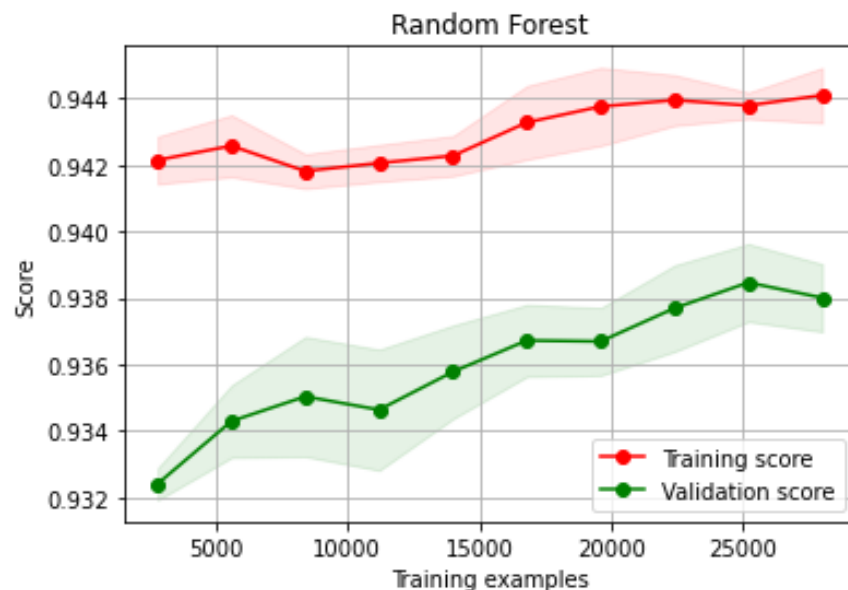
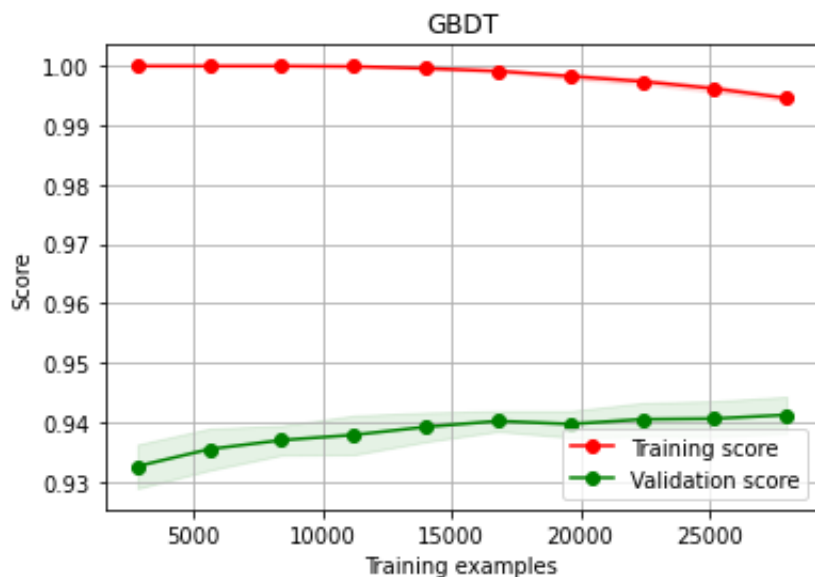


```
'''梯度提升树训练'''
gbdt.fit(train[features], train['labels'])
gbdt_train_score = gbdt.score(train[features], train['labels']) #计算训练集精度
gbdt_test_score = gbdt.score(test[features], test['labels']) #计算测试集精度
print(f'GBDT模型精度: 训练集: {gbdt_train_score:.3f}; 测试集: {gbdt_test_score:.3f}')
```



试一试

(4) 绘制学习曲线进行可视化（注意纵轴是精度，不是误差的大小）



可以看到在这个任务上，梯度提升决策树获得了比随机森林更高的预测精度，且在测试集上仍然表现出了很好的泛化性能。一般而言，基于Boosting的集成学习器可以获得比Bagging集成学习器更高的预测精度，但同时也**存在更高的过拟合风险**。